

ExClique: An Express Consensus Algorithm for High-Speed Transaction Process in Blockchains

Chonghe Zhao¹, Yipeng Zhou², Shengli Zhang¹, Quan Z. Sheng², Yang Zhang³, Shiting Wen⁴

¹College of Electronic and Information Engineering, Shenzhen University, Shenzhen, China

²School of Computing, Macquarie University, Sydney, Australia

³Anuradha and Vikas Sinha Department of Data Science, University of North Texas, Denton, Texas, USA

⁴School of Computer and Data Engineering, NingboTech University, Ningbo, China

Email: zhaochonghe_szu@163.com, yipeng.zhou@mq.edu.au, zsl@szu.edu.cn

michael.sheng@mq.edu.au, yang.zhang@unt.edu, wensht@nbt.edu.cn

Abstract—Proof of Authority (PoA) plays a pivotal role in blockchains for reaching consensus. Clique, which selects consensus nodes to generate blocks with a pre-determined order, is the most popular implementation of PoA due to its low communication overhead and energy consumption. However, our study unveils that the speed to process transactions by Clique is severely restricted by 1) the long communication delay of full blocks (each containing a certain number of transactions) between consensus nodes; and 2) occurrences of no-turn blocks, generated by no-turn nodes if an in-turn block generation fails. Consequently, Clique struggles to support distributed applications requiring a high transaction processing speed, *e.g.*, online gaming. To overcome this deficiency, we propose an Express Clique (ExClique) algorithm by improving Clique from two perspectives: compacting blocks for broadcasting to shorten communication delay and prohibiting the occurrences of no-turn blocks. For performance evaluation, we implement ExClique by modifying Geth of Ethereum, the software implementing Clique, and deploy a permissioned blockchain network by using container technology. The experimental results show that ExClique achieves a substantial enhancement in transactions per second (TPS). Specifically, it boosts TPS by $2.25\times$ in a typical network with 21 consensus nodes and an impressive $7.01\times$ in a large-scale network with 101 consensus nodes when compared to Clique.

Index Terms—Blockchain, Proof-of-Authority, Clique Consensus Algorithm, TPS, Block Broadcasting

I. INTRODUCTION

Blockchain is a distributed ledger system in which participants collectively share information in a decentralized and secure manner. It was first proposed in Bitcoin for decentralized asset transfer [1] and further extended in Ethereum for decentralized applications by introducing smart contracts [2]. Due to its notable characteristics in decentralization, immutability, and anonymity, blockchain has become a disruptive technology in the fields of Web 3.0 [3], [4], Decentralized Finance [5], Non-Fungible Token [6], IoT [7], and Metaverse [8].

Since blockchain is a fully distributed system, consensus algorithms largely determine its security and efficiency. The

consensus algorithm in blockchain plays a role in maintaining a consistent ledger among multiple nodes. To date, various consensus algorithms have been proposed for blockchains [9]–[12], including Proof-of-Work (PoW), Proof-of-Stake (PoS), Practical Byzantine Fault Tolerant (PBFT), and Proof-of-Authority (PoA). Among them, PoA, which selects a set of authorized nodes to produce blocks in turn, gradually stands out as the dominant one due to its enhanced fault tolerance, reduced communication overhead, and energy efficiency [13].

Clique [14], as a mainstream implementation of PoA, has been widely adopted in enterprises, such as Huobi, Binance, and Amazon [15]–[17]. In Clique, participating nodes follow a pre-determined order to process transactions and generate blocks. According to this order, an in-turn (consensus) node is responsible for generating the in-turn block within a specified time interval, *i.e.*, a step. To avoid the single point failure of any in-turn node, about 50% consensus nodes act as no-turn nodes trying to generate no-turn blocks to process transactions if they fail to receive and verify the in-turn block by waiting for a random period of time [18], [19]. Thanks to this simple yet effective design, Clique is incredibly cheap to run and maintain, contributing to its widespread popularity [20].

Despite its prevalence, the optimization of TPS (transactions per second) in Clique has received little attention in existing works, though TPS is one of the most important metrics to gauge the capability of blockchains [21], [22]. Sondhi et al. [23] applied chaos engineering to demonstrate the better ability of Clique to handle load than PBFT and Raft while working in a faulty environment. The work in [24] evaluated transaction confirmation time and TPS performance between different consensus algorithms to verify the suitability of Clique for blockchain-based auditing systems. Islam et al. [13] pointed out that fork is a main factor impacting TPS in Clique without proposing a solution. Due to the absence of TPS optimization in Clique, current blockchains adopting Clique still face the challenge of satisfying applications with massive transactions, *e.g.*, online gaming and payment [25], [26].

In this work, to optimize TPS in Clique, we conduct a comprehensive analysis of Clique. Our study reveals that the transaction processing speed of Clique is restricted by the long communication delay of full block broadcasting and

The corresponding authors are Shengli Zhang and Shiting Wen. The research is supported in part by National Natural Science Foundation of China (62171291), in part by Shenzhen Key Research Project (JSGG20220831095603007), in part by Zhejiang Provincial Natural Science Foundation of China (No. LY24F020021), in part by Ningbo Science and Technology Special Projects (No. 2022Z095, No. 2022Z235), and in part by Australian Research Council (ARC) Discovery Project (DP210101723).

occurrences of no-turn blocks. On the one hand, a Clique block containing a certain number of transactions should be broadcast to all consensus nodes for confirmation. Thus, a long broadcasting delay restricts Clique's capability to generate blocks. On the other hand, in a given time interval, an in-turn block should be generated by the in-turn node for others to accept this block. Yet, any failure to produce and transmit this block in time can lead to the creation of no-turn blocks by other nodes which prolongs block processing time and even results in empty blocks. These issues are detrimental to the TPS performance of blockchains adopting Clique.

To address these challenges, we propose the Express Clique (ExClique) algorithm, which can significantly improve TPS from two perspectives. *First*, we propose a novel compact block broadcast protocol, which can shorten the broadcasting delay of blocks by averting the transmission of redundant information. *Second*, we explore the underlying reason for generating no-turn blocks, based on which we design two means to prohibit no-turn blocks. Specifically, we propose a tighter delay range and a differential order (in lieu of the pre-determined order) to dynamically designate in-turn nodes. The tighter delay range optimizes the random waiting time of no-turn nodes, reducing occurrences of forks and no-turn blocks in Clique. The differential order to configure in-turn nodes effectively prevents the ripple effect of no-turn nodes.¹ Besides, a smart contract is introduced to eliminate unfairness among consensus nodes when using the differential order.

To evaluate the performance of ExClique, we implement it by modifying Geth of Ethereum and deploy ExClique on a permissioned blockchain network by using container technology. The experimental findings unequivocally demonstrate the exceptional performance of ExClique. In a typical blockchain network comprising 21 consensus nodes, ExClique substantially shortens block broadcasting time by a factor of 5, remarkably mitigates fork rates by a factor of 12.4, and enhances TPS by a factor of 2.25. Significantly, TPS gains are further amplified, reaching a remarkable $7.01\times$ improvement in a larger blockchain network with 101 consensus nodes.

The rest of this paper is organized as follows. Section II discusses related works. Section III introduces the background of Clique. The TPS performance of Clique is analyzed in Section IV, which is further optimized in Section V. Section VI describes the experimental design and reports performance evaluation. Finally, Section VII concludes this work.

II. RELATED WORK

Clique, as the most widely used consensus algorithm in blockchain, has attracted huge attention from both academia and industry. However, existing works primarily explored the TPS performance of Clique through comparative evaluation.

In a comprehensive analysis of Clique's performance, Angelis *et al.* [27] first compared its performance with PBFT and Aura using the framework of the CAP Theorem. They

¹Once a no-turn block emerges, the pre-determined order of in-turn nodes will be disrupted, yielding consecutive no-turn blocks in subsequent time intervals. This is called the ripple effect in Clique.

highlighted that Clique achieves the shortest latency for committing a block among three algorithms, potentially resulting in high TPS performance. Similarly, two other works evaluated Clique's performance by selecting PBFT as a baseline: 1) Sondhi *et al.* [23] utilized chaos engineering to assess various blockchain consensus algorithms, finding that Clique surpasses PBFT and Raft, while consistently maintaining favorable TPS even in faulty environments; 2) Ahmad *et al.* [24] analyzed latency in transaction confirmation and TPS performance across different consensus algorithms, including PBFT, PoS, and Clique, within permissioned blockchains. This evaluation confirmed Clique's suitability for blockchain-based auditing systems. Additionally, like [27] selecting Aura as a baseline, the works [13], [24] conducted a comparative analysis between Clique and Aura to evaluate their security, TPS and block period. Their experimental findings revealed that Clique excels in availability and speed but lags in terms of security and consistency due to fork occurrences. These studies, focusing on Clique's performance evaluation, consistently highlight Clique's strength in achieving high TPS performance.

The aforementioned studies shed light on the critical role of TPS in Clique. However, there are multiple potential factors impairing Clique's TPS, *e.g.*, deadlocks. The studies [28], [29] reported that there is a random probability of generating two forked chains with the same weight, causing a deadlock in Clique, despite no malicious nodes trying to attack the network. Once a deadlock occurs, the blockchain systems adopting Clique fail to process transactions, and the TPS is thereby reduced to zero. To address this deficiency, Symons *et al.* [30] proposed a monitoring mechanism capable of detecting and reverting the system state when a deadlock arises.

Yet, none of the existing works systematically investigated all factors impairing TPS. Besides, they failed to propose specific schemes to boost Clique's TPS performance. These gaps will be filled by our work by proposing ExClique.

III. BACKGROUND OF CLIQUE

In this section, we introduce the basics of Clique and the process for generating in-turn and no-turn blocks by Clique.

A. Consensus Nodes

In the initial phase of Clique, the system master authorizes n consensus nodes (indexed from 1 to n) to generate and verify blocks [2], [14]. Normally, each block is generated in a fixed time interval, a.k.a., step, denoted by t_b . In each step, n consensus nodes play three different roles as follows:

- **In-turn Node:** A particular consensus node is designated as the in-turn node that generates the in-turn block for this step with the highest priority.
- **No-turn Nodes:** Multiple consensus nodes act as no-turn nodes which can generate the no-turn blocks with a lower priority if the in-turn consensus node fails.
- **Forbidden Nodes:** The remaining consensus nodes are forbidden nodes that only receive and verify the blocks.

Clique sets the number of consensus nodes for each type based on the following principle. To tolerate up to $\frac{n}{2}$ faulty

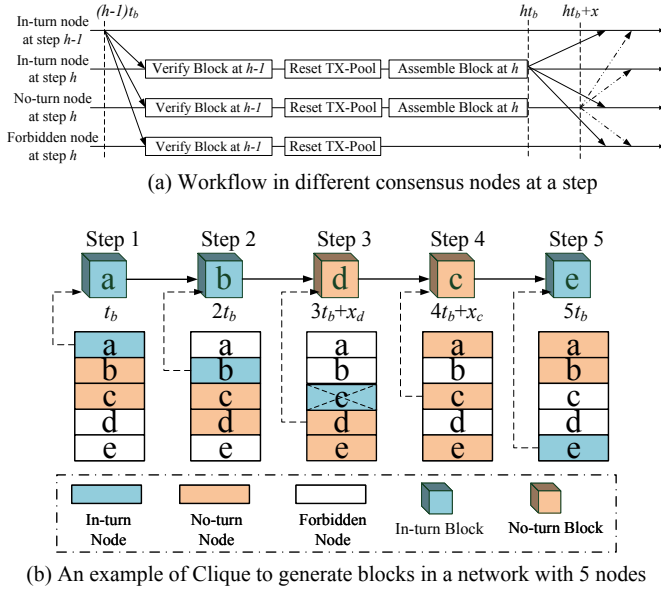


Fig. 1. Specific Design of Clique: (a) Workflow in different consensus nodes; (b) An example to generate blocks.

nodes out of n total consensus nodes, each consensus node can only generate a block (either in-turn block or no-turn block) within $\lfloor \frac{n}{2} \rfloor + 1$ steps [2]. At Step h , Clique sets the consensus node with index $(h+1)\%n$ as the in-turn node, and $\lfloor \frac{n+1}{2} \rfloor - 1$ other consensus nodes (who do not generate any block within the past $\lfloor \frac{n}{2} \rfloor$ steps) as no-turn nodes. The rest of the consensus nodes are set as forbidden nodes.

Suppose that all nodes can play their functions properly, the tasks conducted by each type of consensus node are presented in Fig. 1(a). The in-turn node at Step $h-1$ broadcasts an in-turn block at time $(h-1)t_b$ to start next Step h . After receiving the in-turn block $h-1$, the consensus nodes perform different tasks according to their roles at Step h . For the in-turn node, it verifies the in-turn block $h-1$, resets its transaction pool (TX-Pool) to remove invalid transactions, assembles a new in-turn block h by selecting valid transactions from its TX-Pool, and broadcasts the new in-turn block h at time ht_b to kick off the next step. For no-turn nodes, they verify the in-turn block $h-1$ and reset their TX-Pools. For forbidden nodes, they only verify the in-turn or no-turn blocks and reset their TX-Pools.

B. Block Generation

However, the block generation process in practice is much more complicated than the case presented in Fig. 1(a) because: 1) it is not guaranteed that all nodes can work reliably and 2) Clique is a purely distributed algorithm in which it is difficult to coordinate multiple nodes for block generation. Thus, it is common that no-turn blocks are generated, especially when Clique needs to process an excessive number of transactions.

To understand how no-turn blocks affect Clique, we present a concrete example in Fig. 1(b). There are 5 consensus nodes in the system to generate blocks with the order from a to e at different steps. Blocks a and b are in-turn blocks generated by corresponding in-turn nodes (nodes a and b) at t_b and $2t_b$,

respectively. At Step 3, the in-turn node c fails to generate the in-turn block. In this case, one of two no-turn nodes (*i.e.*, node d and node e) will spontaneously generate no-turn blocks by waiting for a random period of time denoted by x . According to [2], x is randomly selected from the range $(0, w)$, where $w = (\lfloor \frac{n}{2} \rfloor + 1) \cdot 500$ milliseconds. Here, we assume that $x_d < x_e$, and then the no-turn block at Step 3 is generated by node d . Moreover, there is a ripple effect for the influence of the no-turn block on the blockchain system. If node d generates the no-turn block at Step 3, it becomes a forbidden node (to meet the restriction that each node can only generate a block in $\lfloor \frac{n}{2} \rfloor + 1$ steps [2]) and cannot play the in-turn node role at Step 4 implying that a no-turn block will be generated again.

IV. TPS ANALYSIS IN CLIQUE

In this section, we first analyze the factors influencing the TPS performance in Clique, and then define our problem to optimize TPS.

A. TPS Formulation

As we have stated in the last section, a block is generated at each step. According to [31], each block contains a certain number of transactions, denoted as m , which is determined by setting a gas limit in the genesis block. Once m is fixed, TPS is determined, which is no more than $\frac{m}{t_b}$ if there is no no-turn block generated.

Nonetheless, as we have discussed in Fig. 1(b), it is not uncommon that an in-turn node fails to generate an in-turn block resulting in the generation of no-turn blocks. Once no-turn blocks are generated, TPS is compromised from the two following aspects:

- The time interval of a step lasts longer than t_b because no-turn nodes wait for additional x time before generating blocks, where x is randomly drawn from $(0, w)$.
- If the previous step lasts much longer than t_b , the consensus nodes may not have sufficient time to generate full blocks in the current step. Instead, an empty block without any transaction is generated.

Based on our discussion, we derive the expected TPS as:

$$\Lambda = p_0 \cdot \Lambda_0 + (1 - p_0) \cdot \Lambda_{0-}. \quad (1)$$

Here, p_0 represents the probability that in-turn blocks are generated at each step, with TPS equal to $\Lambda_0 = \frac{m}{t_b}$. Λ_{0-} denotes the expected TPS when no-turn blocks are generated, which is very complicated for analysis. Next, we analyze Λ_0 and Λ_{0-} by modeling the block generation process with four cases, as shown in Fig. 2. In this context, Λ_{0-} is further decomposed into Λ_1 , Λ_2 and Λ_3 corresponding to Exceptional Cases 1, 2, and 3, respectively, in Fig. 2.

Normal Case: If an in-turn block is generated at Step h containing m transactions, it implies that the in-turn node completes the following operations within t_b time: 1) broadcasting the in-turn block to all nodes in $b_{h-1}(m)$ time; 2) verifying the in-turn block on all consensus nodes within $v_{h-1}(m)$ time; 3) resetting TX-Pool within $r_h(m)$ time; and 4) assembling the

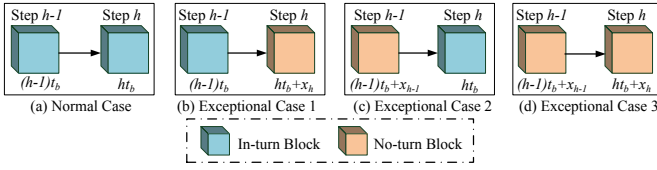


Fig. 2. Four cases at a single step to affect the TPS performance in Clique. block at step h within $a_h(m)$ time. That is, the total time must be no more than the time interval of a step. Hence, we have

$$f_h(m) = b_{h-1}(m) + v_{h-1}(m) + r_h(m) + a_h(m) \leq t_b. \quad (2)$$

Here, $f_h(m)$ is the total time cost to generate the in-turn block and m is a tuneable parameter.

To improve TPS, we should set m as large as possible. Thus, the maximum value of m is

$$m^* = \arg \max_m f_h(m) \leq t_b. \quad (3)$$

It implies that the TPS in the normal case is limited by

$$\Lambda_0 = \frac{m^*}{t_b}. \quad (4)$$

Exceptional Case 1: As shown in Fig. 2(b), this exceptional case occurs when the in-turn node fails to generate an in-turn block at Step h following an in-turn block at Step $h-1$. In this case, $\lfloor \frac{n+1}{2} \rfloor - 1$ no-turn nodes will compete to generate a no-turn block. Since the waiting time of each no-turn node obeys the uniform distribution $U(0, w)$, the expected minimum waiting time of these no-turn nodes (x_h) is the expected minimum value of $\lfloor \frac{n+1}{2} \rfloor - 1$ uniform random variables obeying $U(0, w)$. This waiting time x_h is further denoted by $\Delta_1 = \frac{w}{\lfloor \frac{n+1}{2} \rfloor}$. Thereby, Λ_1 is

$$\Lambda_1 = \frac{m^*}{t_b + \Delta_1}. \quad (5)$$

Given that t_b and w are usually set to $t_b = 3$ seconds and $w = (\lfloor \frac{n}{2} \rfloor + 1) \cdot 500$ milliseconds [2], we have $\Delta_1 \approx 500$ milliseconds and the occurrence of Exceptional Case 1 can roughly lower TPS by 14.3%.

Exceptional Case 2: As shown in Fig. 2(c), Exceptional Case 2 occurs when an in-turn block h is generated following a no-turn block generated at Step $h-1$. As we have stated, when a no-turn block is generated at Step $h-1$, the duration of Step $h-1$ lasts longer than t_b . However, Step h will be over at time ht_b exactly, which implies that the in-turn node at Step h does not have sufficient time to process m transactions (*i.e.*, $ht_b - (h-1)t_b - x_{h-1} < t_b$), and hence an empty in-turn block is generated without any transaction. Thus, we have $\Lambda_2 = 0$.

Exceptional Case 3: As shown in Fig. 2(d), Exceptional Case 3 occurs when two consecutive blocks are generated by no-turn nodes. Let δ_{h-1} and δ_h denote the additional waiting times at Step $h-1$ and Step h , respectively (*i.e.*, $\delta_{h-1} = x_{h-1}$ and $\delta_h = x_h$). These are random variables that follow the same distribution, specifically the minimum waiting time of $\lfloor \frac{n+1}{2} \rfloor - 1$ uniform random variables distributed as $U(0, w)$. Exceptional Case 3 can be analyzed with two separate scenarios: $\delta_h \geq \delta_{h-1}$ and $\delta_h < \delta_{h-1}$.

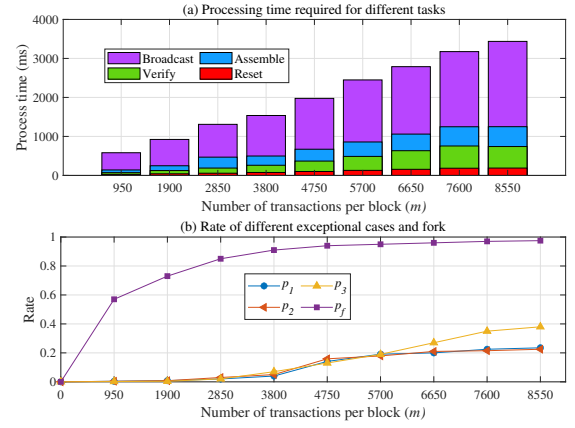


Fig. 3. Investigation Results on Clique: (a) Processing time required for each task; (b) Rate of different exceptional cases and fork.

If $\delta_h \geq \delta_{h-1}$, it implies that no-turn nodes at Step h have sufficient time to process m^* transactions. TPS in this scenario can be expressed as

$$\Lambda_{31} = \frac{m^*}{t_b + \mathbb{E}[\delta_h - \delta_{h-1} | \delta_h \geq \delta_{h-1}]}. \quad (6)$$

Here, $\mathbb{E}$ denotes expectation.

If $\delta_h < \delta_{h-1}$, it implies that no-turn nodes at Step h do not have enough time to process m^* transactions, implying that $\Lambda_{32} = 0$, similar to Exceptional Case 2.

Since δ_h and δ_{h-1} have an identical distribution, the odds of each scenario is 50%. Thus, $\Lambda_3 = \frac{1}{2} \cdot \frac{m^*}{t_b + \mathbb{E}[\delta_h - \delta_{h-1} | \delta_h \geq \delta_{h-1}]}$.

B. Problem Statement

By wrapping up the TPS performance of all cases, the expected TPS of Clique is presented as follows:

$$\begin{aligned} \Lambda &= p_0 \cdot \Lambda_0 + p_1 \cdot \Lambda_1 + p_2 \cdot \Lambda_2 + p_3 \cdot \Lambda_3 \\ &= p_0 \cdot \Lambda_0 + p_1 \cdot \frac{\Lambda_0}{1 + \frac{\Delta_1}{t_b}} + \frac{p_3}{2} \cdot \frac{\Lambda_0}{1 + \frac{\mathbb{E}[\delta_h - \delta_{h-1} | \delta_h \geq \delta_{h-1}]}{t_b}}. \end{aligned} \quad (7)$$

It is easy to verify that $\Lambda_1, \Lambda_2, \Lambda_3 < \Lambda_0$. From Eq. (7), we can see that Λ can be optimized by maximizing Λ_0 and minimizing p_1, p_2 and p_3 (equivalent to maximizing p_0). In the next section, we will explore how to maximize Λ in ExClique.

V. OPTIMIZING TPS IN EXCLIQUE

In this section, we introduce the design of the ExClique algorithm in order to optimize TPS, including a proactive compact block (PCB) protocol to maximize Λ_0 , an accurate delay range to minimize p_1 and p_2 , and a differential order of in-turn nodes to minimize p_3 .

A. Maximizing Λ_0

According to Eqs. (2) and (4), the TPS of the normal case, denoted as Λ_0 , is determined by the total time cost of four tasks: *block broadcasting*, *block verification*, *TX-Pool reset*, and *block assembly*. We improve Λ_0 by 1) identifying the block broadcasting task as the most critical one dominating the total time cost and 2) developing a PCB protocol to shrink communication traffic so as to diminish the total time cost.

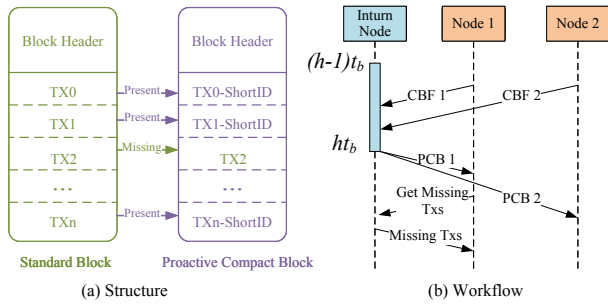


Fig. 4. Detailed Design of PCB: (a) structure; (b) workflow.

Main factor dominating Λ_0 : To identify the main factor that constrains Λ_0 , we conduct the experiment with a permissioned blockchain network comprising 21 consensus nodes executing Clique. In this experiment, we set the step duration time $t_b = 3$ seconds. To test the TPS performance, we vary the number of transactions contained in each block from 950 to 8,550.² In Fig. 3(a), we present our experimental results with the x-axis representing the number of transactions per block and the y-axis representing the time cost of each block. In addition, we decompose the time cost into four parts corresponding to the time cost of each task in Eq. (2).

From the experimental result, we can observe that the total time cost and the time cost of each task rise with respect to the increase of m , and notably, the time cost of block broadcasting dominates the total time cost. This issue arises from the rapid increase in standard block size driven by the surge in the number of transactions [32]. Therefore, Λ_0 can be substantially improved if we can effectively reduce the time cost of block broadcasting.

PCB for speeding up block broadcasting: To speed up block broadcasting, we design a proactive compact block (PCB) protocol. The design of PCB is based on the fact that the consensus nodes frequently exchange transactions with each other which actually already exist in their TX-Pools. We use a counting bloom filter (CBF) [33] to indicate the missing status of each transaction on each consensus node. By exchanging CBFs prior to transaction broadcasting, the consensus nodes can avoid broadcasting those redundant transactions which are already owned by receiving nodes.

It has been investigated in Bitcoin that nodes exchange transactions in their TX-Pools before these transactions are packaged in a block, and thus the similarity of their TX-Pools is very high [34]. To improve the block broadcasting speed in Bitcoin, a compact block protocol was proposed [35], [36], in which transactions in a standard block are represented by their hashes. Receiving nodes can identify these transactions from their local TX-Pools by these hashes.

Unfortunately, it is infeasible to straightly apply the compact block protocol in blockchain systems adopting Clique in which the period to generate a block is much shorter. As a result, nodes in Clique have a very low similarity of TX-Pools, which can excessively incur additional communication rounds for

broadcasting missing transactions [32]. For example, the probability of incurring additional communication rounds exceeds 90% in Ethereum Mainnet. It implies that the efficiency of the compact block protocol will significantly deteriorate in blockchains adopting Clique [32], [37].

Since the in-turn node and the time allocated for generating in-turn blocks at each step can be easily identified in blockchains adopting the PoA consensus algorithm (whether Clique with a fixed order or our ExClique with a differential order, as discussed in Section V-C), we design PCB by introducing CBF which avoids incurring extra communication rounds in most cases. As illustrated in Fig. 4(a), each transaction within a PCB block is represented as either “missing” or “present” based on whether it exists in the receiving node’s TX-Pool or not. If a transaction is tagged as “missing”, the complete transaction data will be included in the PCB block without the need for additional communication rounds. Conversely, for transactions tagged as “present”, only a short hash (a 6-byte short ID, as depicted in Fig. 4(a)) is embedded into the PCB block to minimize broadcasting overload.

To tag transactions, each consensus node maintains a CBF. This data structure allows a consensus node to represent all transactions within its local TX-Pool concisely, enabling the instant detection of missing transactions. Besides, unlike the traditional Bloom Filter that can only support the *add* operation, CBF supports both *add* and *remove* operations so that it can be updated dynamically [38]. In other words, an element can be added or removed when a new transaction is stored or an invalid transaction is removed in TX-Pool, *e.g.*, the transaction that has been packaged in a block. Due to this dynamic maintenance feature, each consensus node can easily create and maintain a CBF.

The detailed workflow of PCB is presented in Fig. 4(b). At Step h , before the birth of the in-turn block in this step, node 1 and node 2 (either forbidden or no-turn nodes) send their respective CBFs to the in-turn node. The in-turn node, through the examination of their CBFs, identifies missing and present transactions for node 1 and node 2, respectively. Then, the in-turn node can tailor different PCB blocks (PCB 1 and PCB 2) for them. These PCB blocks are then broadcasted, with PCB 1 being sent to node 1 and PCB 2 to node 2. Upon the reception of their respective PCB blocks, node 1 and node 2 can reconstruct the standard block by retrieving transactions from their TX-Pools.

Although the chance is rare, it is worth noting that using CBF in ExClique may also consume additional communication rounds for broadcasting missing transactions. First, CBF cannot guarantee the uniqueness of the hash ID for each transaction, *i.e.*, the false negative problem in CBF [39]. If hash IDs conflict, CBF cannot exactly tell missing and present transactions in receiving nodes’ TX-Pools. Second, transactions in the blockchain are dynamic. It will consume additional communication rounds if CBF cannot factor in all changes of transactions in time, *e.g.*, removal of a transaction in TX-Pools after sending CBF but before receiving PCB.

Compression rate of PCB: Intuitively speaking, how much

²Other parameters of this experiment are the same as those in Section VI-A. For more detailed parameter settings, please refer to Section VI-A.

time cost can be reduced by PCB is dependent on the number of short transactions that can be represented by hash values (6-byte short IDs). In contrast, the smallest traffic consumed by a full transaction for token transfer [40] is around 110 Bytes. It implies that the compression rate is at least $110/6 \approx 18$ times by replacing a full transaction with a short ID in PCB.

The fraction of short transactions is related to the network size. In the Ethereum MainNet with thousands of nodes [32], this fraction is around 0.9. For the blockchain network adopting Clique, the network has only dozens of nodes and the fraction of short transactions exceeds 0.9. Therefore, our PCB protocol can considerably shrink communication traffic, and hence diminish the time cost of block broadcasting.

B. Minimizing p_1 and p_2

We minimize p_1 and p_2 by using the minimization of the fork probability in Clique as the surrogate objective because forks are caused by the generation of no-turn blocks which then trigger Exceptional Cases 1 and 2. Clique adopts the parallel block generation mechanism to effectively prevent the single-point failure of in-turn nodes from havoc on the entire network. Nonetheless, the cost of the parallel block generation mechanism lies in the occurrence of forks. Typically, if the in-turn block cannot be generated in time, no-turn nodes will try to generate no-turn blocks, which potentially causes forks. Once forks occur, it is possible that Exceptional Cases 1 and 2 described in the last section will occur.

Forks influencing p_1 and p_2 : To demonstrate the relationship between forks and the two probabilities (p_1 and p_2), we measure the fork event probability (denoted by p_f) together with p_1 and p_2 in the experiment presented in Fig. 3(a). Then, we plot p_f , p_1 and p_2 v.s. the number of transactions in each block, i.e., m , in Fig. 3(b).

The experimental results show that: 1) with the increase of m in each block, the fork probability p_f rapidly rises; 2) both p_1 and p_2 have a strong positive correlation with p_f ; 3) when $p_f > 0.8$, we can see a significant increase of p_1 and p_2 . Therefore, it is effective to use the minimization of p_f as a surrogate objective for minimizing p_1 and p_2 .

Modeling fork probability: According to the description in the last section, the in-turn node at Step $h-1$ broadcasts the in-turn block at time $(h-1)t_b$, while a no-turn node also broadcasts its no-turn block if it cannot receive and verify the in-turn block successfully until time $(h-1)t_b + x$. In other words, a fork will occur at a step when the random delay x is smaller than the total time spent on broadcasting and verifying the in-turn block. Since x follows the uniform distribution $U(0, w)$, the probability of a fork incurred by a single no-turn node at Step h is denoted by

$$p_s = \frac{b_{h-1}(m) + v_{h-1}(m)}{w}. \quad (8)$$

Considering that there are $\lfloor \frac{n+1}{2} \rfloor - 1$ no-turn nodes at Step h to generate no-turn blocks, the probability of a fork incurred by multiple no-turn nodes is thereby denoted by

$$p_f = 1 - (1 - p_s)^{\lfloor \frac{n+1}{2} \rfloor - 1}. \quad (9)$$

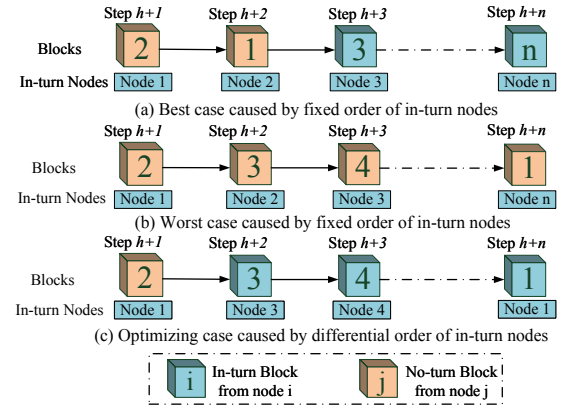


Fig. 5. Different cases at multiple steps caused by different orders of in-turn nodes when a no-turn block is committed.

Accurate delay range to prohibit forks: According to Eq. (8), a fork likely occurs when a no-turn node fails to consider the time required for the in-turn block broadcasting and validation (i.e., $b_h(m) + v_h(m)$). In other words, when the delay x , randomly selected by a no-turn node from the range of $(0, w)$ is less than the time cost for broadcasting and validation of the in-turn block, a fork occurs.

To take this delay into account, we change the range of the random delay x chosen by no-turn nodes to (β, w) . The fork probability after this change is

$$p'_f = 1 - (1 - \frac{b_h(m) + v_h(m) - \beta}{w})^{\lfloor \frac{n+1}{2} \rfloor - 1}. \quad (10)$$

According to Eq. (10), the ideal case is $p'_f = 0$ if we can set $b_h(m) + v_h(m) = \beta$. Each no-turn node can approximately measure $b_h(m)$ and $v_h(m)$ at the current Step h based on the broadcasting and validation time cost of the last in-turn block generated from the same consensus node. Using the measured $b_h(m)$ and $v_h(m)$, each no-turn node dynamically set $U(\beta, w)$ as the distribution to generate the random delay x .

C. Minimizing p_3

Exceptional Case 3 may occur following the occurrence of a no-turn block from Exceptional Case 1. Due to the rigidly fixed order of in-turn nodes in Clique, there exists a ripple effect for occurrences of Exceptional Case 3. It means that a no-turn block committed at a step will trigger more no-turn blocks at the subsequent steps. Thus, Exceptional Case 3 occurs more frequently than Exceptional Cases 1 and 2. This conjecture is verified by the experiment presented in Fig. 3(b), in which p_3 grows faster than p_1 and p_2 with m .

Fixed order of in-turn nodes influencing p_3 : To better understand the ripple effect in Clique due to the fixed/pre-determined order of in-turn nodes, we present two representative cases in Fig. 5(a) and Fig. 5(b), in which the order of in-turn nodes is 1, 2, ..., n . Fig. 5(a) presents the case without the ripple effect because the in-turn block is generated at Step $h+3$ following Exceptional Case 1 at Step $h+1$ and Exceptional Case 2 at Step $h+2$. In contrast, Fig. 5(b) shows the case with a significant ripple effect. In this case, a series of no-turn blocks are generated over multiple consecutive steps.

By scrutinizing the difference between these two cases, we can see the reason for the ripple effect. Clique restricts that each node can only generate a single block over $\lfloor \frac{n}{2} \rfloor + 1$ steps [2], [14]. Node 3 has committed to generate a no-turn block at Step $h + 2$, and thus it cannot generate the in-turn block at Step $h + 3$, resulting in the generation of a no-turn block from node 4 at Step $h + 3$. However, a specific node that will commit to the generation of the no-turn block at Step $h + 2$ cannot be centrally controlled, implying that the influence of the ripple effect can persist over multiple steps.

Differential order of in-turn nodes: To prohibit the ripple effect in Clique, we propose a differential order of in-turn nodes. In Clique, the in-turn node at Step h is specified to the consensus node with index i satisfying $h \% n = i$, where n is the total number of consensus nodes. Rather than fixing indices of in-turn nodes, ExClique dynamically sets the order of in-turn nodes as follows. Without loss of generality, we consider the scenario that in-turn node i at Step $h + 1$ fails to generate an in-turn block, and no-turn node j is the winner to generate a no-turn block committed to the global ledger. The in-turn node at the subsequent Step $h + 2$ is then assigned to the consensus node with index $j + 1$. In other words, the index of the in-turn node at each step is purely determined by the index of the consensus node generating the block (regardless of no-turn or in-turn blocks) in the last step.

Fig. 5(c) exemplifies the differential order of in-turn nodes in ExClique. If in-turn node 1 fails at Step $h + 1$ and no-turn node 2 is the winner at this step, node 3 becomes the in-turn node at Step $h + 2$, and consensus node 4 becomes the in-turn node at Step $h + 3$, and so on.

The differential order of ExClique introduces unfairness among consensus nodes because it can disrupt the income balance of different nodes. Specifically, when $j > i + 1$, consensus nodes with indexes falling within the range $[i + 1, j)$ will miss the opportunity to earn transaction fees (*i.e.*, block rewards) by generating blocks during Steps $[h + i + 1, h + i + n - j]$. Here, we suppose that j is the no-turn node generating the no-turn block after node i generates a no-turn block in the last step.

Fair Smart Contract: To eliminate the unfairness caused by using the differential order in Clique, we propose a smart contract to distribute cumulative transaction fees equally among multiple consensus nodes, *e.g.*, a distributed contract used in BNB blockchain [16]. In other words, rather than sending transaction fees directly to block generators, transaction fees are temporarily accumulated within the fair smart contract. Then, the fees are distributed equally to active consensus nodes after a certain period.

Nonetheless, the smart contract for evenly distributing transaction fees should have a mechanism to prevent inactive or lazy nodes. Otherwise, inactive or lazy nodes can earn fees without generating any blocks. Besides, the smart contract takes a longer time to settle fees, which might be unacceptable for nodes who are eager to access funds promptly [16]. Considering these issues, our design is based on a sliding window with length n (the same as the number of nodes) and adopts the mechanism of uncle block in Ethereum [41] to

Algorithm 1: Fair Smart Contract

Require: transaction fees f for a confirmed block at Step h , number of consensus nodes n , two empty arrays A_1 and A_2

Ensure: A_2

```

1:  $i = 0$ 
2: while  $i < n$  do
3:   Clear  $A_1$ 
4:   Select the block generator at Step  $h - n + i$  and add it in  $A_1$ 
5:   Detect all uncle blocks recorded in the confirmed block at Step  $h - n + i$  and append their generators in  $A_1$ 
6:   Calculate the number of generators in  $A_1$  and denote it as  $a_1$ 
7:    $j = 0$ 
8:   while  $j < a_1$  do
9:     if  $A_1[j]$  is not in  $A_2$  then
10:      Append  $A_1[j]$  in  $A_2$ 
11:     end if
12:      $j = j + 1$ 
13:   end while
14:    $i = i + 1$ 
15: end while
16: Calculate the number of generators in  $A_2$  and denote it as  $a_2$ 
17: Transfer the transaction fees  $f/a_2$  to each generator in  $A_2$ 

```

detect lazy nodes. The fees are only distributed to active nodes periodically. The detailed design is presented in *Algorithm 1*. Specifically, a block is considered as a confirmed block if it is committed to the blockchain, and as an uncle block if it is not committed but contains correct information and is recorded in the confirmed block. Thus, a consensus node can be deemed active when it generates the uncle block even if its block is not committed. Then, for a given Step h , consensus nodes not generating any confirmed/uncle blocks in the past n steps³ are identified as lazy/inactive nodes. Lastly, the smart contract only transfers the rewards evenly to active nodes.

VI. PERFORMANCE EVALUATION

In this section, we implement ExClique and deploy it in a permissioned blockchain network to evaluate its performance.

A. Experimental Settings

To verify and evaluate the performance of ExClique, we implement the prototype of different Clique consensus algorithms by modifying Geth, which is the blockchain software used for implementing Clique [2].

1) *System Settings:* For conducting our experiments, we build a permissioned blockchain network environment using the docker container technology on a Linux server. In this environment, the server deploys multiple docker containers and allocates its hardware resources (*e.g.*, CPU, RAM, and SSD) to containers. Each docker container is designated as a virtual “physical machine” representing a location somewhere in the world, and network parameters between two containers are configured by geographical labels to simulate a distributed network. The specific blockchain software is loaded and executed within a docker container, serving as a consensus node. These consensus nodes then establish connections with each other to form a permissioned blockchain network. The specific parameters of the permissioned blockchain network are shown in Table I.

³Each node has at least one chance to become an in-turn node within every n steps based on the limit of block generation and the differential order.

TABLE I
PARAMETERS OF PERMISSIONED BLOCKCHAIN NETWORK

Parameter	Values
Network Size n	21, 31, 41, 51, and 101
Step Duration t_b	3s
Topology	Fully P2P Network
Bandwidth	32 Mbit/s
Packet Loss Rate	Random Value within (0, 0.1)
Propagation Delay	Random Value within (0, 200ms)

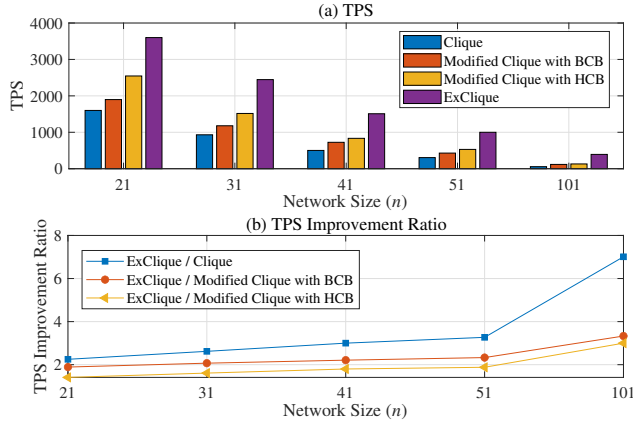


Fig. 6. TPS performance of different Cliques under different network sizes.

2) *Baselines*: We compare the performance of ExClique with three baselines: 1) Clique; 2) Modified Clique with BCB, *i.e.*, Clique plus the compact block protocol that replaces full transactions with transaction hashes [36]; 3) Modified Clique with HCB, *i.e.*, Clique plus hybrid compact block that replaces full transactions with short transaction hashes and embeds a prediction model into missing transactions to reduce additional communication delay [32].

3) *Evaluation Metrics*: To evaluate the performance of ExClique, we select the following metrics: 1) TPS performance to evaluate the speed to process transactions; 2) block size and block broadcasting time to verify the efficiency of PCB; 3) forks rate (p_f) and the rate of each exceptional case (p_1 , p_2 , and p_3) to verify the efficiency improvement by the accurate delay range and the differential order of in-turn nodes; and 4) block reward distribution on consensus nodes to verify the efficiency of the fair smart contract.

B. Experimental Results

TPS Performance: The network size, denoted as n , is a critical parameter significantly influencing TPS performance because it is difficult to reach a consensus when there are a large number of consensus nodes. For evaluating TPS, we vary the network size from 21 to 101 to study how much TPS can be improved by ExClique under different network scales. Given n , to find the optimal TPS for each consensus algorithm, we enumerate different values for m to search m^* satisfying Eq. (3) to ensure that we compare the optimal TPS of different consensus algorithms.

The experimental results are presented in Fig. 6, from which we can see that: 1) ExClique always achieves the highest TPS performance in comparison to other baselines, indicating

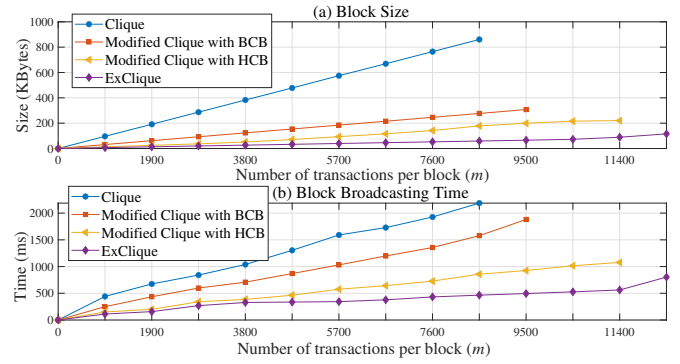


Fig. 7. Block Size and Block Broadcasting Time for different Cliques in the network with 21 consensus nodes.

its advantages in applications with massive transactions. 2) Compared with baselines, Clique achieves a higher TPS gain with a larger network scale. In particular, when the network size is 21 (which is the size adopted in [15], [16]), Clique can improve TPS by $1.41 \times - 3.0 \times$ compared to baselines. However, when the network scale is increased to 101, the improvement is increased to $2.25 \times - 7.01 \times$. 3) Modified Clique with BCB and Modified Clique with HCB are two novel competitive baselines proposed in our work. They outperform Clique by compacting blocks. However, their performance is inferior to ours since ExClique designs the compact block protocol based on the characteristics of Clique-based blockchains and adopts additional strategies to prohibit exceptional cases.

Block Size and Block Broadcasting Time: We conduct experiments to verify that ExClique can improve TPS because PCB in ExClique can shrink the block size effectively to reduce the block broadcasting time. In the experiment, we measure the average block size and block broadcasting time of each algorithm in the network with 21 consensus nodes. Here, the network size aligns with the configurations in real-world systems, *e.g.*, [15], [16]. We vary m from 0 to 12,350 per block to exhibit that PCB can consistently outperform baselines under various scenarios.

The experimental results are shown in Fig. 7, where the horizontal axis represents the number of transactions per block (*i.e.*, m). Fig. 7 exhibits the superb performance of the PCB protocol in ExClique. Notably, regardless of m , ExClique always achieves the smallest block size and shortest block broadcasting time among all four consensus algorithms. Besides, the improvement of ExClique becomes more pronounced as m rises. When $m = 8,550$, the block size in ExClique is only around $1/14$ that of Clique, $1/5$ that of Modified Clique with BCB, and $1/3$ that of Modified Clique with HCB. Similarly, the block broadcasting time of ExClique is approximately $1/5$ that of Clique, $1/3$ that of Modified Clique with BCB, and $5/9$ that of Modified Clique with HCB. In Fig. 7, it is worth noting that the block size can significantly restrict the transaction process capacity of the blockchain. When $m = 12,350$, ExClique can still work normally. In contrast, the system using Clique fails to work when $m > 8,550$. Modified Clique with BCB and Modified Clique with HCB fail to work when $m > 9,500$ and $m > 11,400$, respectively. These results underscore the

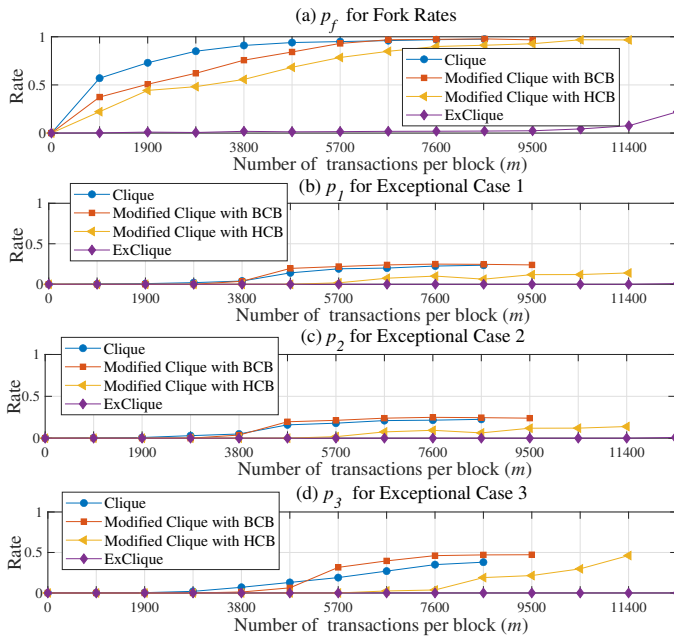


Fig. 8. Rate of different exceptional cases and forks for different Cliques in the network with 21 consensus nodes.

remarkable efficiency gains achieved by our PCB in ExClique. **Rate of Different Exceptional Cases and Fork:** Our ExClique employs an accurate delay range to reduce fork rates p_f to alleviate the occurrences of Exceptional Cases 1 and 2 (i.e., decrease p_1 and p_2), and uses the differential order of in-turn nodes to avoid Exceptional Case 3 (i.e., $p_3 = 0$). To illustrate the effectiveness of these two mechanisms, we measured p_f , p_1 , p_2 , and p_3 in the typical blockchain network with 21 consensus nodes by varying different m .

The experimental results are presented in Fig. 8, with the horizontal axis representing the number of transactions per block (m). From Fig. 8(a), the result reveals a significant difference in p_f between our ExClique and other baselines. For baselines, their rates p_f surge rapidly with respect to m , and all exceed 0.5 when m reaches 5,700. Due to their high p_f , it is not difficult to understand that exceptional cases will occur more frequently with higher p_1 and p_2 , which has been shown in Fig. 8(b) and Fig. 8(c). Conversely, in ExClique, the forks rate remains under 0.1, even when m is set to 11,400, which can greatly reduce the occurrence odds of Exceptional Cases 1 and 2. Meanwhile, as shown in Fig. 8(d), with the help of the differential order of in-turn nodes, the ripple effect causing consecutive occurrences of Exceptional Case 3 is completely averted, making p_3 always equal to 0. Therefore, our ExClique can considerably enhance TPS by effectively prohibiting occurrences of exceptional cases.

Besides, it is worth noting that prior investigations [37], [42] have consistently reported fork rates less than 0.1 in prominent blockchain networks such as Bitcoin, Ethereum 1.0, Litecoin, and Dogecoin. This steadfast low fork rate has been instrumental in upholding these networks' robust security and unwavering stability. Thus, our ExClique offers a robust solution that maintains security even as m is extremely large,

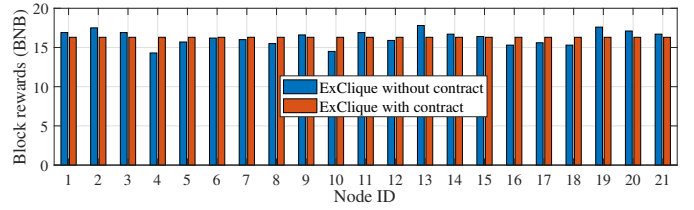


Fig. 9. Block rewards earned by each node in the network with 21 consensus nodes when ExClique adopts or not adopt the fair smart contract.

e.g., 11,400, ultimately improving TPS. In contrast, baselines struggle to achieve high system security when $m > 950$, highlighting the perspective of ExClique in practice.

Block Reward Distribution on Consensus Nodes: As discussed in Section V-C, the differential order of ExClique can cause unfairness among consensus nodes, disrupting the income balance across different nodes. To address this issue, we design a fair smart contract. To validate the effectiveness of our method, we measure block rewards for each consensus node in a typical network with 21 consensus nodes, comparing ExClique with/without the smart contract. In each experiment case, 2,100 blocks are generated, mirroring the block reward distribution of the BNB blockchain [16].

The experimental results are shown in Fig. 9, with the x-axis representing node ID. We can observe that ExClique, when adopting the fair smart contract, maintains a more balanced income distribution among different nodes compared to the case without the smart contract. The results shed light on that implementing the fair smart contract can effectively mitigate unfairness caused by the differential order. In contrast, if the fair smart contract is absent, nodes had significant variations in block rewards. This validates that the fair smart contract effectively addresses the income imbalance and enhances the fairness of block reward distribution in ExClique.

VII. CONCLUSION

Clique stands as a prominent implementation of PoA. It was designed with a focus on energy efficiency, rendering it highly suitable for deployment in blockchain networks. However, the low transaction processing speed in large Clique has been overlooked in existing works, which has significantly restricted the use of Clique with massive transactions. Our work amends this deficiency by proposing ExClique, which improves TPS performance by designing: 1) a proactive compact block protocol to minimize the delay of block broadcasting based on the predictable in-turn node at each step; 2) a tighter delay range to diminish the fork rate; and 3) a differential order of in-turn nodes to prohibit no-turn block occurrences. Importantly, ExClique is friendly for implementation and compatible with the current blockchain systems adopting Clique. At last, extensive experiments are conducted to verify the superiority of ExClique, which reduces the fork rate to almost 0. Moreover, it substantially improves TPS by $2.25\times$ in a blockchain network with 21 consensus nodes and an impressive $7.01\times$ in a larger network with 101 consensus nodes. The superb performance of ExClique can advance blockchains by widening their applications in large-scale networks.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized Business Review*, 2008.
- [2] (2015) Go-ethereum. <https://github.com/ethereum/go-ethereum>.
- [3] L. V. Kiong, *Web3 Made Easy: A Comprehensive Guide to Web3: Everything you need to know about Web3, Blockchain, DeFi, Metaverse, NFT and GameFi*. Liew Voon Kiong, 2022.
- [4] T. Wang, S. Zhang, Q. Yang, and S. C. Liew, "Account service network: A unified decentralized web 3.0 portal with credible anonymity," *IEEE Network*, vol. 37, no. 6, pp. 101–108, 2023.
- [5] K. Qin, L. Zhou, and A. Gervais, "Quantifying blockchain extractable value: How dark is the forest?" in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 198–214.
- [6] L. Ante, "Non-fungible token (nft) markets on the ethereum blockchain: Temporal development, cointegration and interrelations," *Economics of Innovation and New Technology*, pp. 1–19, 2022.
- [7] F. Chen, J. Wang, C. Jiang, T. Xiang, and Y. Yang, "Blockchain based non-repudiable iot data trading: Simpler, faster, and cheaper," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022, pp. 1958–1967.
- [8] H. Duan, J. Li, S. Fan, Z. Lin, X. Wu, and W. Cai, "Metaverse for social good: A university campus prototype," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 153–161.
- [9] Q. Wang and R. Li, "A weak consensus algorithm and its application to high-performance blockchain," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021, pp. 1–10.
- [10] L. Shi, T. Wang, J. Li, S. Zhang, and S. Guo, "Pooling is not favorable: Decentralize mining power of pow blockchain using age-of-work," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2756–2769, 2022.
- [11] J. Xu, C. Wang, and X. Jia, "A survey of blockchain consensus protocols," *ACM Comput. Surv.*, vol. 55, no. 13s, jul 2023. [Online]. Available: <https://doi.org/10.1145/3579845>
- [12] D. P. Oyinloye, J. S. Teh, N. Jamil, and M. Alawida, "Blockchain consensus: An overview of alternative protocols," *Symmetry*, vol. 13, no. 8, 2021. [Online]. Available: <https://www.mdpi.com/2073-8994/13/8/1363>
- [13] M. M. Islam, M. M. Merlec, and H. P. In, "A comparative analysis of proof-of-authority consensus algorithms: Aura vs clique," in *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, 2022, pp. 327–332.
- [14] P. Szilágyi. (2017) Eip-225: Clique proof-of-authority consensus protocol. <https://eips.ethereum.org/EIPS/eip-225>.
- [15] (2020) Heco chain. <https://github.com/stars-labs/heco-chain>.
- [16] Binance. (2024) Binance smart chain. <https://docs.bnbchain.org/docs/learn/intro/>.
- [17] A. Flores. (2018) Launch enterprise-ready blockchain networks on aws in minutes with kaleido—a consensys solution. <https://aws.amazon.com/cn/blogs/apn/>.
- [18] E. Parinya, G. Vincent, and J. Guillaume, "The attack of the clones against proof-of-authority," in *Network and Distributed Systems Security (NDSS) Symposium*, 2020, pp. 1–14.
- [19] Q. Wang, R. Li, Q. Wang, S. Chen, and Y. Xiang, "Exploring unfairness on proof of authority: Order manipulation attacks and remedies," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 123–137.
- [20] c. wackerow, minimalism and 0x00Z, "Proof-of-authority (poa)." [Online]. Available: <https://ethereum.org/zh/developers/docs/consensus-mechanisms/poa/>
- [21] S. Gupta, J. Hellings, S. Rahnama, and M. Sadoghi, "Building high throughput permissioned blockchain fabrics: challenges and opportunities," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, 2020.
- [22] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "Blockbench: A framework for analyzing private blockchains," in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 1085–1100.
- [23] S. Sondhi, S. Saad, K. Shi, M. Mamun, and I. Traore, "Chaos engineering for understanding consensus algorithms performance in permissioned blockchains," in *2021 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*. IEEE, 2021, pp. 51–59.
- [24] A. Ahmad, A. Alabduljabbar, M. Saad, D. Nyang, J. Kim, and D. Mohaisen, "Empirically comparing the performance of blockchain's consensus algorithms," *IET Blockchain*, vol. 1, no. 1, pp. 56–64, 2021. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/blc2.12007>
- [25] L. Xiao, "Blockchain in online games and what can be learned from it," *Frontiers in Business, Economics and Management*, vol. 12, no. 3, pp. 35–38, 2023.
- [26] D. Agrawal, A. El Abbadi, M. J. Amiri, S. Maiyya, and V. Zakhary, "Blockchains and databases: Opportunities and challenges for the permissioned and the permissionless," in *Advances in Databases and Information Systems: 24th European Conference, ADBIS 2020, Lyon, France, August 25–27, 2020, Proceedings 24*. Springer, 2020, pp. 3–7.
- [27] S. D. Angelis, L. Aniello, R. Baldoni, F. Lombardi, A. Margheri, and V. Sassone, "PBFT vs proof-of-authority: Applying the CAP theorem to permissioned blockchain," in *ITASEC*, ser. CEUR Workshop Proceedings, vol. 2058. CEUR-WS.org, 2018.
- [28] C. N. Samuel, S. Glock, F. Verdier, and P. Guitton-Ouhamou, "Choice of ethereum clients for private blockchain: Assessment from proof of authority perspective," in *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2021, pp. 1–5.
- [29] M. Martínez. (2019) Poa network, all the sealers are waiting for each other after 2 months running, possible deadlock? <https://github.com/ethereum/go-ethereum/issues/18402>.
- [30] A. Symons. (2021) Deadlock resolver. https://github.com/IDChain-eth/IDChain/blob/release/1.9/deadlock_resolver.py.
- [31] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014. [Online]. Available: <https://cryptodeep.ru/doc/paper.pdf>
- [32] C. Zhao, T. Wang, S. Zhang, and S. C. Liew, "Hcb: Enabling compact block in ethereum network with secondary pool and transaction prediction," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 1, pp. 1077–1092, 2024.
- [33] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: a scalable wide-area web cache sharing protocol," *IEEE/ACM Transactions on Networking*, vol. 8, no. 3, pp. 281–293, 2000.
- [34] K. Dae-Yong, E. Meryam, and J. Hongtaek, "Examining bitcoin mem-pools resemblance using jaccard similarity index," in *2020 21st Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 2020, pp. 287–290.
- [35] Z. Hu and Z. Xiao, "Dino: A block transmission protocol with low bandwidth consumption and propagation latency," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022, pp. 1319–1328.
- [36] M. Corallo. (2016) Compact block relay. <https://github.com/bitcoin/bips/blob/master/bip-0152.mediawiki>.
- [37] C. Zhao, S. Zhang, T. Wang, and S. C. Liew, "Bodyless block propagation: Tps fully scalable blockchain with pre-validation," *Future Generation Computer Systems*, vol. 163, p. 107516, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24004801>
- [38] A. Broder and M. Mitzenmacher, "Network applications of bloom filters: A survey," *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [39] D. Guo, Y. Liu, X. Li, and P. Yang, "False negative problem of counting bloom filter," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 5, pp. 651–664, 2010.
- [40] J. K. lumierre, Linmao Song. (2018) Transaction size. <https://ethereum.stackexchange.com/questions/30175/>.
- [41] Y. Huang, B. Wang, and Y. Wang, "Research and application of smart contract based on ethereum blockchain," *Journal of Physics: Conference Series*, vol. 1748, no. 4, p. 042016, jan 2021. [Online]. Available: <https://dx.doi.org/10.1088/1742-6596/1748/4/042016>
- [42] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 3–16.